



Modélisation de l'ergodicité par des transformations d'images



MELLI Léo - 38551

Plan

Projet : Étude de l'ergodicité de transformations d'images en observant leurs comportements lorsque le nombre d'itérations et les dimensions de l'image varient

I. Introduction et définitions

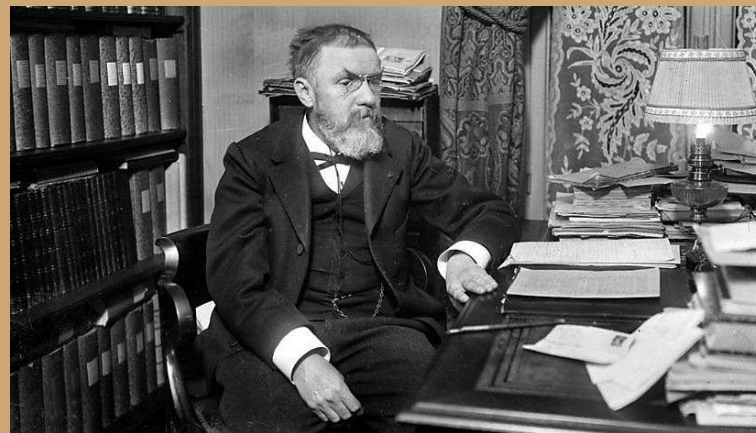
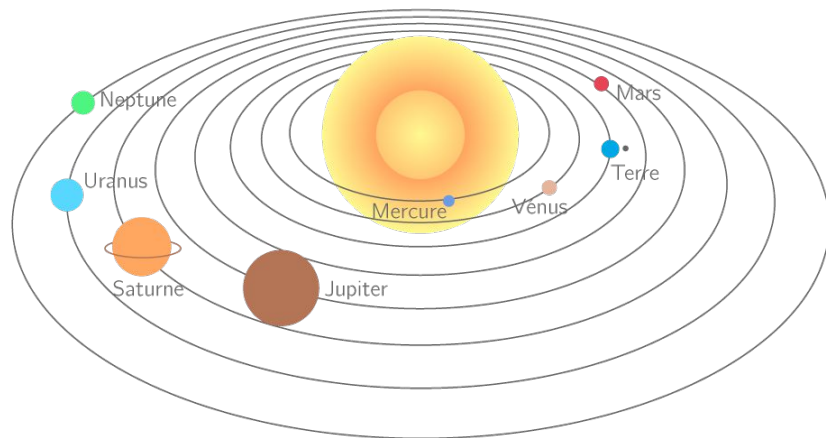
- Présentation historique de la théorie ergodique
- Définitions et théorèmes

II. Les transformations bijectives étudiées

- La transformation du Boulanger
- La transformation du Photomaton
- La transformation combinée

I. Introduction et définitions

Contexte historique



Henri Poincaré
(1854-1912)

Georges David
Birkhoff (1884-1944)

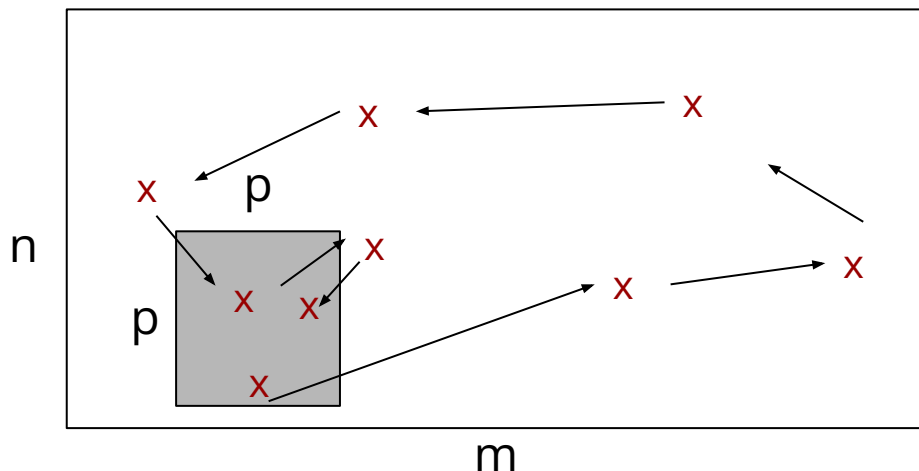


Théorème de récurrence de Poincaré

Soit X un plan,

Si $f : X \rightarrow X$ est une fonction continue bijective et conserve les surfaces, alors à partir d'un certain nombre de composition/ itérations, presque tous les points de la fonction donnée retournent une infinité de fois aussi proche que l'on veut de sa position d'origine.

Illustration:



Ergodicité

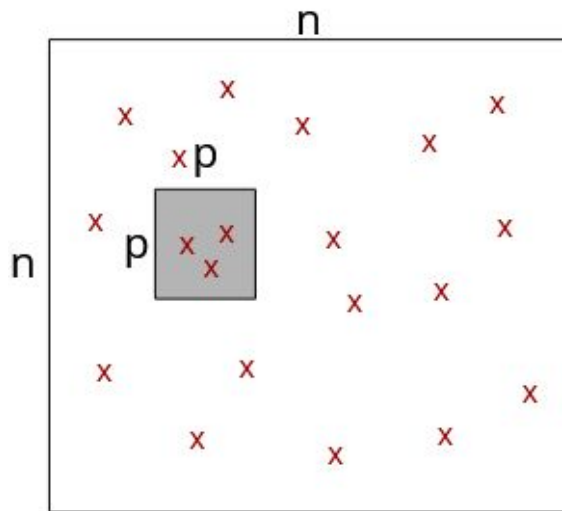
Définition: *L'ergodicité traduit l'idée qu'un point d'un système va éventuellement visiter tout l'espace traversé par le système de façon uniforme et aléatoire.*

Théorème ergodique de Birkhoff

Soit D un plan et $f : D \rightarrow D$ une fonction bijective, pour presque tout x_0 appartenant à D ,

$$\forall B \in D, \lim_{n \rightarrow +\infty} \frac{1}{n+1} \text{Card}(B \cap \{f^{(k)}(x_0), k \leq n\}) = \frac{A(B)}{A(D)}$$

Illustration:



II. Les transformations

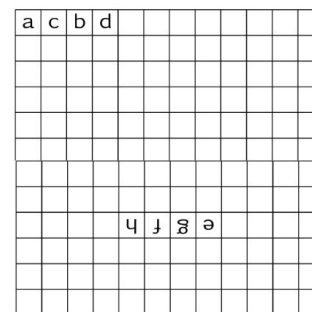
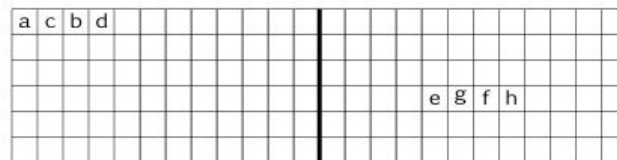
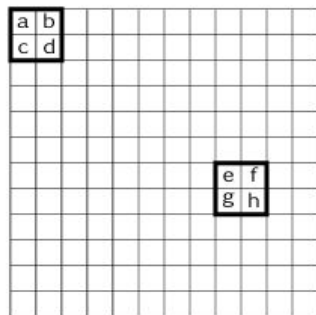
La transformation du Boulanger

pour une image de taille $p \times q$:

$$x_1 = \left\lfloor \frac{x}{2} \right\rfloor$$

$$y_1 = 2y + (x \bmod 2)$$

$$(x', y') = \begin{cases} (x_1, y_1) & \text{si } y_1 < q \\ (p - 1 - x_1, 2q - 1 - y_1) & \text{sinon} \end{cases}$$



La transformation du Photomaton

pour une image de taille $p \times q$:

$$(x', y') = \begin{cases} (x/2, y/2) & \text{si } x \text{ et } y \text{ sont pairs} \\ (x/2, \lfloor y/2 \rfloor + q/2) & \text{si } x \text{ est pair et } y \text{ impair} \\ (\lfloor x/2 \rfloor + p/2, y/2) & \text{si } x \text{ est impair et } y \text{ pair} \\ (\lfloor x/2 \rfloor + p/2, \lfloor y/2 \rfloor + q/2) & \text{si } x \text{ et } y \text{ sont impairs} \end{cases}$$

si x et y sont pairs

si x est pair et y impair

si x est impair et y pair

si x et y sont impairs

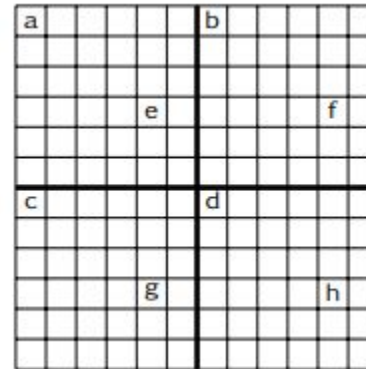
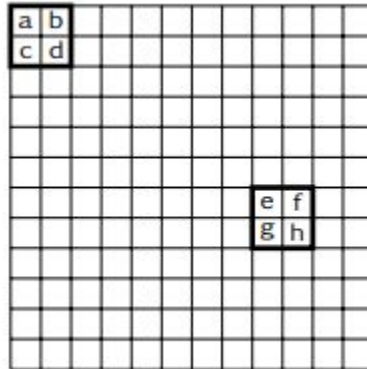


Image d'étude

hypothèses:

On considère que la transformation est ergodique si, pour 100 images par seconde, la période de l'image est supérieure à 100 ans; la période doit donc être supérieure à 3.15×10^{11}

dimensions

prises

initialement

carrée:

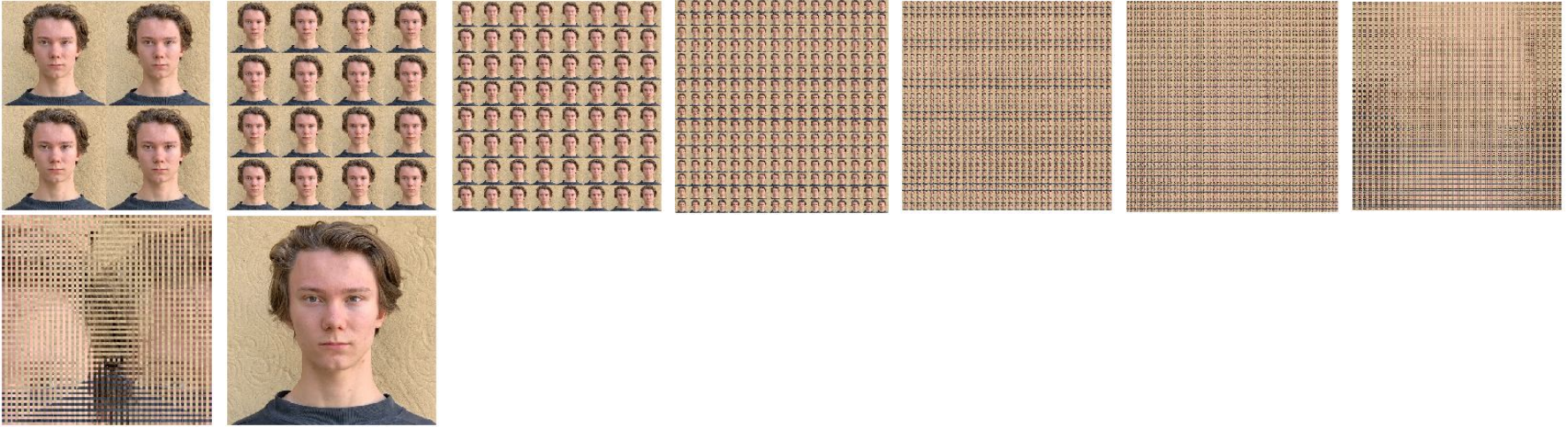
- 512 x 512
- 451 x 451

rectangulaire:

- 201 x 301
- 200 x 300



Illustration



Itérations de 1 à 9 pour l'image de taille 512 x 512

Le programme ici applique la transformation du photomaton à l'image et affiche son évolution à chaque itération. Il s'arrête lorsque tous les pixels sont revenus à leur position d'origine.

Résultats

Le programme retourne la période de l'image, et la liste des périodes des pixels dans l'image. La période est le PPCM des éléments de la liste des périodes des pixels

<u>Dimensions</u>	512 x 512	200 x 300	451 x 451	201 x 301
<u>Résultats</u>	(9, [1, 9, 3])	(396, [1, 99, 132, 396, 11, 12])	PAS DE PÉRIODE	PAS DE PÉRIODE

Transformation alternée BP

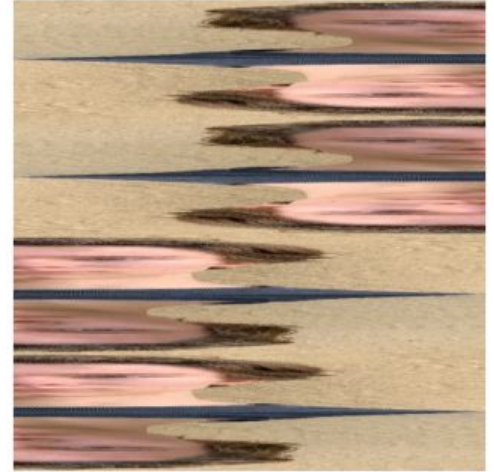
On choisit de réaliser la transformation du boulanger, puis du photomaton... et ainsi de suite



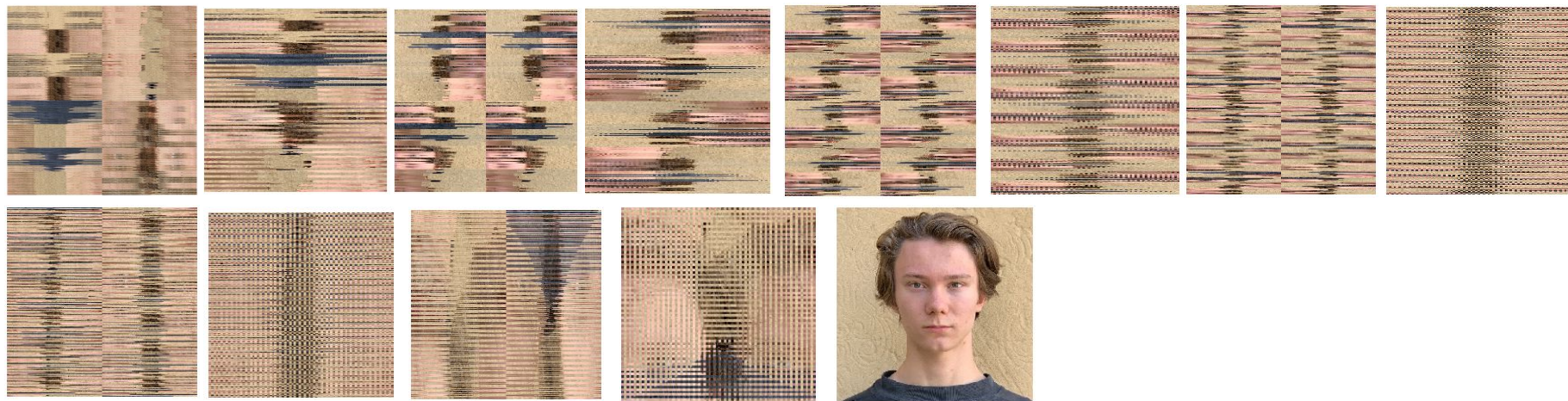
1ère itération



2nde itération



3ème itération



itérations de 10 à 22, pour l'image 512 x 512

période pour l'image de taille 512 x 512 : (4290, [1, 22, 2, 15, 13])

Résultats

<u>Dimensions</u>	512 x 512	451 x 451	200 x 300	201 x 301
<u>Période</u>	4290	PAS DE PÉRIODE	$1.4 * 10^9$	PAS DE PÉRIODE
<u>Tableau des périodes</u>	[1,22,2,15,13]		[1,446,2,36,444,9 4,40,50,20,16]	

Autres Résultats

<u>Dimensions</u>	64 x 64	128 x 128	256 x 256	512 x 512	1024 x 1024	2048 x 2048
<u>Période</u>	570	18	24	4290	25830	26
<u>Tableau des périodes</u>	[1, 10, 2, 19, 30, 6]	[1, 18, 2, 6]	[1, 12, 2, 24, 8, 6, 4]	[1,22,2,15,13]	[1, 14, 2, 63, 70, 10, 41, 7]	[1, 26, 2]

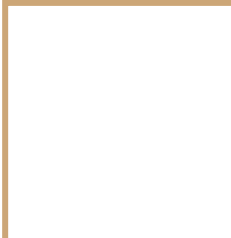
Transformation combinée avec une variable aléatoire

On fixe une probabilité p de réaliser la fonction boullanger

<u>Dimensions</u>	512 x 512	200 x 300
<u>Exemple de période pour $p = 0.2$</u>	$4.9 \cdot 10^{150}$	$1.7 \cdot 10^{30}$
<u>Exemple de période pour $p = 0.85$</u>	$3.7 \cdot 10^{266}$	$7.3 \cdot 10^{29}$

Tableau récapitulatif

<u>Ergodique?</u>	PAIRE CARRÉE	IMPAIRE CARRÉE	PAIRE RECTANGULAIRE	IMPAIRE RECTANGULAIRE
<u>Dimensions</u>	512 x 512	451 x 451	200 x 300	201 x 301
<u>Bou langer</u>	NON	NA	OUI	NA
<u>Photomaton</u>	NON	NA	NON	NA
<u>Combinée alternée</u>	NON	NA	NON	NA
<u>Combinée avec une probabilité</u>	OUI	NA	OUI	NA



Annexe



Bibliographie

[1] Luís BARREIRA et Claudia VALLS : *Théorie des systèmes dynamiques : une introduction* : EDP Sciences

[2] Étienne GHYS : *Variations autour du théorème de récurrence de Poincaré* :
<https://perso.ens-lyon.fr/ghys/articles/recurrence.pdf>

[3] Jean Paul DELAHAYE : *Images brouillées, images retrouvées* :
<https://criuniv.univ-lille.fr/~jdelahay/dnalor/ImagesBrouillees.pdf>

[4] François BÉGUIN : *Le théorème de récurrence de Poincaré* :
<https://images-archive.math.cnrs.fr/Le-theoreme-de-recurrence-de-Poincare.html#nh13>

[5] George D. BIRKHOFF : *Proof of the ergodic théorème* : PNAS, Volume 17, 1931

Fonction Boulanger

```
def fonction_boulanger(x, y, p, q):  
    x1, y1 = x // 2, 2 * y + x % 2  
    if y1 < q:  
        return x1, y1  
    else:  
        return p - 1 - x1, 2 * q - 1 - y1  
  
def boulanger(img):  
    p, q = img.shape[0], img.shape[1]  
    img2 = np.empty_like(img)  
    for x in range(p):  
        for y in range(q):  
            img2[fonction_boulanger(x, y, p, q)] = img[x, y]  
    return img2
```

Fonction Photomaton

```
def fonction_photomaton(x, y, n, p):  
    if x % 2 == 0 and y % 2 == 0:  
        return x//2, y//2  
    if x%2 == 1 and y%2 == 0:  
        return x//2+n//2,y//2  
    if x%2 == 0 and y%2 == 1:  
        return x//2,y//2+p//2  
    if x%2 == 1 and y%2 == 1:  
        return x//2+n//2,y//2+p//2  
  
def photomaton(img):  
    p, q = img.shape[0], img.shape[1]  
    img2 = np.empty_like(img)  
    for x in range(p):  
        for y in range(q):  
            img2[fonction_photomaton(x, y, p, q)] = img[x, y]  
    return img2
```

Fonction combinée alternée

```
def fonction_transformation(x, y, p, q):  
    global compteur  
    if compteur%2 == 0:  
        F = fonction_boulangier(x, y, p, q)  
    else:  
        F = fonction_photomaton(x, y, p, q)  
  
    compteur += 1  
    return F  
  
def transformation(img):  
    global compteur  
    if compteur%2 == 0:  
        F = boulangier(img)  
    else:  
        F = photomaton(img)  
    compteur+= 1  
    return F
```

Fonction combinée avec une probabilité p

```
def fonction_transformation(x, y, p, q):  
    if rd.random() < 0.2:  
        return fonction_boulangier(x, y, p, q)  
    else:  
        return fonction_photomaton(x, y, p, q)  
  
def transformation(img):  
    if rd.random() < 0.2:  
        return boulangier(img)  
    else:  
        return photomaton(img)
```

Fonction tableau des périodes + période

```
def periode_pixel(x, y, p, q):  
    n = 1  
    x1, y1 = fonction_transformation(x, y, p, q)  
    while (x1, y1) != (x, y):  
        n += 1  
        x1, y1 = fonction_transformation(x1, y1, p, q)  
    return n
```

```
def tableau_des_periodes(img):  
    p, q = img.shape[0], img.shape[1]  
    r = np.zeros((p, q), dtype=int)  
    for x in range(p):  
        for y in range(q):  
            if r[x, y] == 0:  
                s = periode_pixel(x, y, p, q)  
                x1, y1 = x, y  
                for _ in range(s):  
                    r[x1, y1] = s  
                    x1, y1 = fonction_transformation(x1, y1, p, q)  
    return r
```

```
def ppcm(a, b):  
    return a * (b // gcd(a, b))
```

```
def periode_transformation(img):  
    p, q = img.shape[0], img.shape[1]  
    r = tableau_des_periodes(img)  
    lst = []  
    for x in range(p):  
        for y in range(q):  
            if r[x, y] not in lst:  
                lst.append(int(r[x, y]))  
  
    m = 1  
    for n in lst:  
        m = ppcm(m, n)  
    return m, lst
```

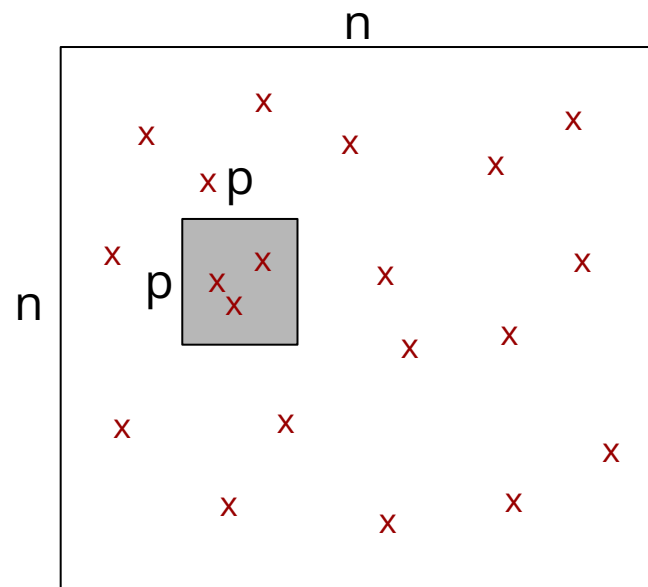
Fonction pour le calcul du nombre de pixels ayant une période de 15 ou 13

```
tab = tableau_des_periodes(img)
L = []
for i in range(len(tab)):
    if 15 in tab[i][:]:
        L.append(i)
    if 13 in tab[i][:]:
        L.append(i)
```

[16, 32, 64, 65, 68, 129, 132, 136, 144, 258, 260, 264, 272, 273, 288, 443, 445, 477, 478, 494, 495, 503, 507]

Résultats pour Boulanger

<u>Dimensions</u>	200 x 300	512 x 512
<u>Période</u>	$1.6 * 10^{18}$	19
<u>Tableau des périodes</u>	[1, 28950, 2687, 10938, 15114, 2226, 24, 28, 14, 9, 5, 4]	[1,19]



x

a	c	b	d									
				u	f	g	e					